

Multiset — Resolución

multiset.hs — occursMSE / filterMSE / isValidMSE / evalMSE / simpMSE / fold / rec

Tipos de datos

```
data MExp a = EmptyMS
            | AddMS a (MExp a)
            | RemoveMS a (MExp a)
            | UnionMS (MExp a) (MExp a)
            | MapMS (a -> a) (MExp a)

instance (Show a) => Show (MExp a) where
  show EmptyMS      = "EmptyMS"
  show (AddMS x xs)  = "AddMS " ++ show x ++ " (" ++ show xs ++ ")"
  show (RemoveMS x xs) = "RemoveMS " ++ show x ++ " (" ++ show xs ++ ")"
  show (UnionMS xs ys) = "UnionMS (" ++ show xs ++ ") (" ++ show ys ++ ")"
  show (MapMS _ xs)  = "MapMS (" ++ show xs ++ ")"
```

Auxiliar: unoSi

```
unoSi :: Bool -> Int
unoSi True = 1
unoSi _    = 0
```

Punto 1.a — occursMSE

```
occursMSE :: Eq a => a -> MExp a -> Int
occursMSE x ms = occursMESWith x id ms

occursMESWith :: Eq a => a -> (a -> a) -> MExp a -> Int
occursMESWith x f EmptyMS      = 0
occursMESWith x f (AddMS y ms)  = unoSi (f y == x) + occursMESWith x f ms
occursMESWith x f (RemoveMS y ms) = max 0 (occursMESWith x f ms - unoSi (x == f y))
occursMESWith x f (UnionMS ms1 ms2) = (occursMESWith x f ms1) + (occursMESWith x f ms2)
occursMESWith x f (MapMS g ms)    = occursMESWith x (f . g) ms
```

Punto 1.b — filterMSE

```
filterMSE :: (a -> Bool) -> MExp a -> MExp a
filterMSE f EmptyMS      = EmptyMS
filterMSE f (AddMS y ms)  = if f y
                           then AddMS y (filterMSE f ms)
                           else filterMSE f ms
filterMSE f (RemoveMS y ms) = if f y
                           then RemoveMS y (filterMSE f ms)
                           else filterMSE f ms
filterMSE f (UnionMS ms1 ms2) = UnionMS (filterMSE f ms1) (filterMSE f ms2)
filterMSE f (MapMS g ms)    = MapMS g (filterMSE (f . g) ms)
```

Punto 1.c — isValidMSE

```
isValidMSE :: Eq a => MSExp a -> Bool
isValidMSE ms = sonTodosPositivos (balance id ms)

sonTodosPositivos :: [(a, Int)] -> Bool
sonTodosPositivos [] = True
sonTodosPositivos ((v, c): ps) = c >= 0 && sonTodosPositivos ps

balance :: Eq a => (a -> a) -> MSExp a -> [(a, Int)]
balance f EmptyMS      = []
balance f (AddMS y ms)  = update (+1) (f y) (balance f ms)
balance f (RemoveMS y ms) = update (subtract 1) (f y) (balance f ms)
balance f (UnionMS ms1 ms2) = (balance f ms1) ++ (balance f ms2)
balance f (MapMS g ms)   = balance (f . g) ms

update :: Eq a => (Int -> Int) -> a -> [(a, Int)] -> [(a, Int)]
update f x []          = [(x, f 0)]
update f x ((v, c): ps) = if v == x
                           then (v, f c) : update f x ps
                           else (v, c) : update f x ps
```

Punto 1.d — evalMSE

```
evalMSE :: Eq a => MSExp a -> [a]
evalMSE EmptyMS      = []
evalMSE (AddMS y ms)  = y : evalMSE ms
evalMSE (RemoveMS y ms) = quitar y (evalMSE ms)
evalMSE (UnionMS ms1 ms2) = (evalMSE ms1) ++ (evalMSE ms2)
evalMSE (MapMS f ms)   = map f (evalMSE ms)

quitar :: Eq a => a -> [a] -> [a]
quitar _ [] = []
quitar e (x: xs) = if x == e then xs else x : quitar e xs
```

Punto 1.e — simpMSE

```
simpMSE :: Eq a => MSExp a -> MSExp a
simpMSE EmptyMS      = EmptyMS
simpMSE (AddMS y ms)  = AddMS y (simpMSE ms)
simpMSE (RemoveMS y ms) = simpRemove y (simpMSE ms)
simpMSE (UnionMS ms1 ms2) = simpUnion (simpMSE ms1) (simpMSE ms2)
simpMSE (MapMS f ms)   = simpMap f (simpMSE ms)

simpRemove :: Eq a => a -> MSExp a -> MSExp a
simpRemove y (AddMS x ms) = if y == x then ms else (RemoveMS y (AddMS x ms))
simpRemove y e            = RemoveMS y e

simpUnion :: MSExp a -> MSExp a -> MSExp a
simpUnion EmptyMS e      = e
simpUnion e EmptyMS      = e
simpUnion e1 e2          = UnionMS e1 e2

simpMap :: (a -> a) -> MSExp a -> MSExp a
simpMap f EmptyMS = EmptyMS
simpMap f e       = MapMS f e
```

Demostración: evalMSE . simpMSE = evalMSE

por principio de extencionalidad, sea mse un multiset cualquiera.

¿ para todo mse. (evalMSE . simpMSE) mse = evalMSE mse ?

por def. de compose, es equivalente a decir:

¿ para todo mse. evalMSE (simpMSE mse) = evalMSE mse ?

por principio de inducción estructural sobre la estructura mse

dem:

casoBase) mse = EmptyMS

¿ evalMSE (simpMSE EmptyMS) = evalMSE EmptyMS ?

casoInductivo1) mse = (AddMS y ms)

HI) ¿ evalMSE (simpMSE ms) = evalMSE ms !

TI) ¿ evalMSE (simpMSE (AddMS y ms)) = evalMSE (AddMS y ms) ?

casoInductivo2) mse = (RemoveMS y ms)

HI) ¿ evalMSE (simpMSE ms) = evalMSE ms !

TI) ¿ evalMSE (simpMSE (RemoveMS y ms)) = evalMSE (RemoveMS y ms) ?

casoInductivo3) mse = (UnionMS ms1 ms2)

HI1) ¿ evalMSE (simpMSE ms1) = evalMSE ms1 !

HI2) ¿ evalMSE (simpMSE ms2) = evalMSE ms2 !

TI) ¿ evalMSE (simpMSE (UnionMS ms1 ms2)) = evalMSE (UnionMS ms1 ms2) ?

casoInductivo4) mse = (MapMS f ms)

HI) ¿ evalMSE (simpMSE ms) = evalMSE ms !

TI) ¿ evalMSE (simpMSE (MapMS f ms)) = evalMSE (MapMS f ms) ?

Demuestro

casoBase)

LI) evalMSE (simpMSE EmptyMS)

= --simpMSE.1
evalMSE EmptyMS

LD) evalMSE EmptyMS

VALE ESTE caso

casoInductivo1)

LI) evalMSE (simpMSE (AddMS y ms))

= -- simpMSE.2
evalMSE AddMS y (simpMSE ms)

= -- evalMSE.2
y : evalMSE (simpMSE ms)

= -- HI
y : evalMSE ms

LD) evalMSE (AddMS y ms)

= -- evalMSE.2
y : evalMSE ms

VALE ESTE caso

casoInductivo2)

```
LI)    evalMSE (simpMSE (RemoveMS y ms))
=
      evalMSE (simpRemove y (simpMSE ms))      -- simpMSE.3
=
      quitar y (evalMSE (simpMSE ms))          -- Lema.1
=
      quitar y (evalMSE ms)                    -- HI

LD)    evalMSE (RemoveMS y ms)
=
      quitar y (evalMSE ms)
```

casoInductivo3)

```
LI)    evalMSE (simpMSE (UnionMS ms1 ms2))
=
      evalMSE (simpUnion (simpMSE ms1) (simpMSE ms2))  --simpMSE.4
=
      evalMSE (simpMSE ms1) ++ evalMSE (simpMSE ms2)  -- LEMA.2
=
      evalMSE ms1 ++ evalMSE ms2                      -- HI1 y HI2

LD)    evalMSE (UnionMS ms1 ms2)
=
      evalMSE e1 ++ evalMSE e2                        -- evalMSE.4
=
```

VAle ESTE caso

LEMA.2

evalMSE (simpUnion m1 m2) = evalMSE m1 ++ evalMSE m2

sea m1, m2, dos multisets cualquiera, quiero ver que:

{evalMSE (simpUnion m1 m2) = evalMSE m1 ++ evalMSE m2?}

por analisis de casos sobre m1' y m2':

c1, m1 = EmptyMS, m2
{evalMSE (simpUnion EmptyMS m2) = evalMSE EmptyMS ++ evalMSE m2?}
c2, m1 != EmptyMS, m2 = EmptyMS
{evalMSE (simpUnion m1 EmptyMS) = evalMSE m1 ++ evalMSE EmptyMS?}
c3, m1 != EmptyMS, m2 != EmptyMS
{evalMSE (simpUnion m1 m2) = evalMSE m1 ++ evalMSE m2?}

c1)

```
LI)    evalMSE (simpUnion EmptyMS m2)
=
      evalMSE m2                                -- simpUnion.1

LD)    evalMSE EmptyMS ++ evalMSE m2
=
      [] ++ evalMSE m2                          --evalMSE.1
=
      evalMSE m2                                -- ++
```

VALE ESTE CASO

c3)

```
LI) evalMSE (simpUnion m1 m2)
=
      evalMSE (UnionMs m1 m2)                  --simpUnion.3
=
      evalMSE m1 ++ evalMSE m2                  --evalMSE.4
```

LD) evalMSE m1 ++ evalMSE m2

VALE ESTE CASO

LEMA.1

```
evalMSE (simpRemove y ms) = quitar y (evalMSE ms)
```

sea ms un multiset cualquiera, quiero ver que:

```
{evalMSE (simpRemove y ms) = quitar y (evalMSE ms)?
```

por analisis de caso sobre ms:

```
c1: ms = AddMS x ms2
```

```
{ evalMSE (simpRemove y (AddMS x ms2)) = quitar y (evalMSE (AddMS x ms2)) ?
```

```
c2: ms != AddMS x ms2
```

```
{ evalMSE (simpRemove y ms) = quitar y (evalMSE ms) ?
```

c1)

```
LI) evalMSE (simpRemove y (AddMS x ms2))
```

```
= simpRemove.1
```

```
evalMSE (if y == x
          then ms
          else (RemoveMS y (AddMS x ms2)))
```

```
= por prop. del if
```

```
(if y == x
  then evalMSE ms
  else evalMSE ((RemoveMS y (AddMS x ms2))))
```

```
= evalMSE.3
```

```
(if y == x
  then evalMSE ms
  else evalMSE (quitar y (evalMSE (AddMS x ms2))))
```

```
= evalMSE.2
```

```
(if y == x
  then evalMSE ms
  else evalMSE (quitar y (x : evalMSE ms2)))
```

```
LD) quitar y (evalMSE (AddMS x ms2))
```

```
= evalMSE.2
```

```
quitar y (x : evalMSE ms2)
```

```
= quitar.2
```

```
if x == y
  then evalMSE ms2
  else x : quitar y (x : evalMSE ms2)
```

incompleto

Resumen

funciones de matter matching -> demostrar por analisis de casos

funciones con if -> demostrar con subcasos sobre if.

foldMSEExp / recMSEExp

foldMSEExp

```
:: b -> (a -> b -> b) -> (a -> b -> b) -> (b -> b -> b)
```

```
-> ((a -> a) -> b -> b) -> MSEExp a -> b
```

```
foldMSEExp b a r u m EmptyMS = b
```

```
foldMSEExp b a r u m (AddMS y ms) = a y (foldMSEExp b a r u m ms)
```

```
foldMSEExp b a r u m (RemoveMS y ms) = r y (foldMSEExp b a r u m ms)
```

```
foldMSEExp b a r u m (UnionMS ms1 ms2) = u (foldMSEExp b a r u m ms1) (foldMSEExp b a r u m ms2)
```

```
foldMSEExp b a r u m (MapMS f ms) = m f (foldMSEExp b a r u m ms)
```

recMSEExp

```
:: b -> (a -> MSEExp a -> b -> b) -> (a -> MSEExp a -> b -> b)
```

```
-> (MSEExp a -> b -> MSEExp a -> b -> b)
```

```
-> ((a -> a) -> MSEExp a -> b -> b) -> MSEExp a -> b
```

```
recMSEExp b a r u m EmptyMS = b
```

```
recMSEExp b a r u m (AddMS y ms) = a y ms (recMSEExp b a r u m ms)
```

```
recMSEExp b a r u m (RemoveMS y ms) = r y ms (recMSEExp b a r u m ms)
```

```
recMSEExp b a r u m (UnionMS ms1 ms2) = u ms1 (recMSEExp b a r u m ms1) ms2 (recMSEExp b a r u m ms2)
```

```
recMSEExp b a r u m (MapMS f ms) = m f ms (recMSEExp b a r u m ms)
```

Versiones con fold

```
occursMESWith' :: Eq a => a -> (a -> a) -> MSExp a -> Int
occursMESWith' x f ms = foldMSExp
  (\x f -> 0)
  (\y rs x f -> unoSi (f y == x) + rs x f)
  (\y rs x f -> max 0 (rs x f - unoSi (f y == x)))
  (\rs1 rs2 x f -> rs1 x f + rs2 x f)
  (\g rs x f -> rs x (f . g))
  ms x f

filterMSE' :: (a -> Bool) -> MSExp a -> MSExp a
filterMSE' f ms = foldMSExp
  (\f -> EmptyMS)
  (\y rs f -> if f y
    then AddMS y (rs f)
    else rs f)
  (\y rs f -> if f y
    then RemoveMS y (rs f)
    else rs f)
  (\rs1 rs2 f -> UnionMS (rs1 f) (rs2 f))
  (\g rs f -> MapMS g (rs (f . g)))
  ms f

balance' :: Eq a => (a -> a) -> MSExp a -> [(a, Int)]
balance' f ms = foldMSExp (\f -> [])
  (\y rs f -> update (+1) (f y) (rs f))
  (\y rs f -> update (subtract 1) (f y) (rs f))
  (\rs1 rs2 f -> rs1 f ++ rs2 f)
  (\g rs f -> rs (f . g))
  ms f

evalMSE' :: Eq a => MSExp a -> [a]
evalMSE' = foldMSExp []
  (\y rs -> y : rs)
  (\y rs -> quitar y rs)
  (\rs1 rs2 -> rs1 ++ rs2)
  (\f rs -> map f rs)

simpMSE' :: Eq a => MSExp a -> MSExp a
simpMSE' = foldMSExp EmptyMS
  (\y rs -> AddMS y rs)
  (\y rs -> simpRemove y rs)
  (\rs1 rs2 -> simpUnion rs1 rs2)
  (\f rs -> simpMap f rs)
```